

Tether

Data model, DDL, relationships, RLS policies, and API map

Version 1.0 Date 17 Sept 2026 Engine PostgreSQL 15 (Supabase) Extensions pgcrypto, pgvector, citext
Companion PRD · TRD · UI/UX

Contents

1. Entity relationship overview

2. Enums and extensions

3. Core identity tables

4. Pairing tables

5. Content tables

6. Daily loop tables
7. AI tables

8. Insight, subscription, safety tables

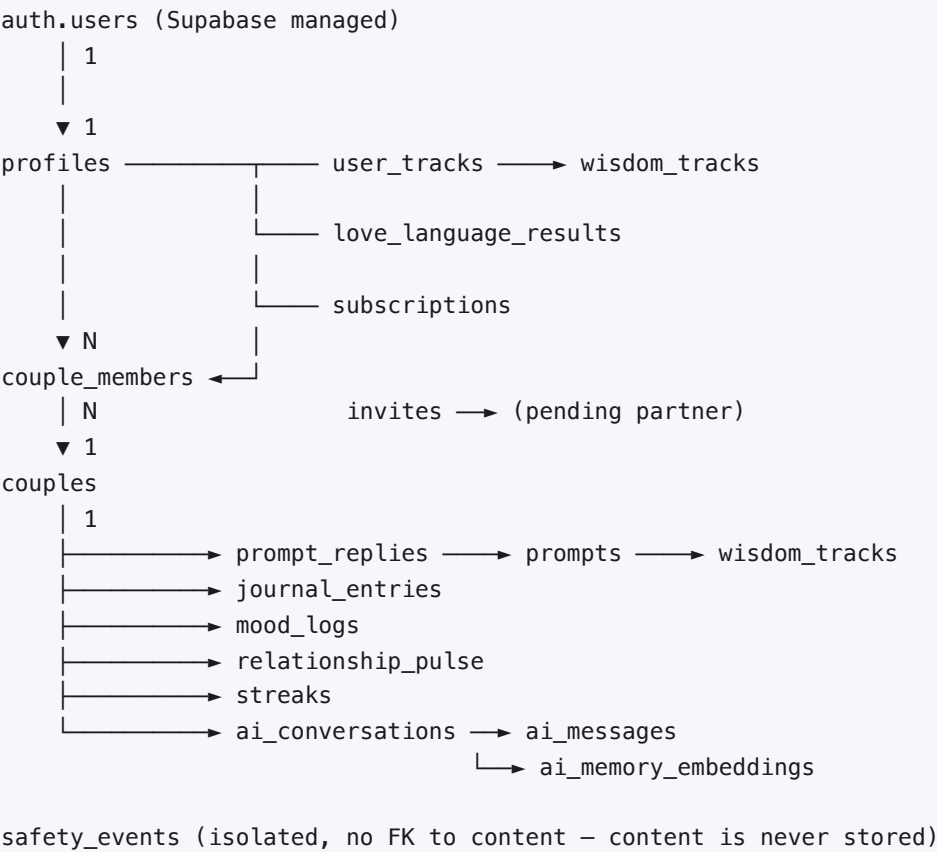
9. Indexes

10. Row Level Security policies

11. API map

12. Migrations and seed data

01 Entity relationship overview



Encryption boundary

Columns marked `BYTEA` hold AES-256-GCM ciphertext produced on device. **The server has no key and cannot read them.** Never add a plaintext column for journal or AI content — not even for debugging.

02 Enums and extensions

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS citext;

CREATE TYPE wisdom_track AS ENUM ('secular','vedic','quranic','biblical');
CREATE TYPE member_role AS ENUM ('partner_a','partner_b');
CREATE TYPE invite_status AS ENUM ('pending','accepted','expired','revoked');
CREATE TYPE invite_method AS ENUM ('link','qr','code');
CREATE TYPE pulse_state AS ENUM ('thriving','drifting','strained','unknown');
CREATE TYPE visibility AS ENUM ('shared','private');
CREATE TYPE entry_source AS ENUM ('prompt','journal','coach');
CREATE TYPE safety_action AS ENUM
('suppressed','resources_shown','escalated','false_positive');
CREATE TYPE entitlement AS ENUM ('free','premium','couples_plus');
```

03 Core identity tables

profiles

```
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  display_name CITEXT NOT NULL,
  avatar_url TEXT,
  avatar_initials CHAR(2),
  locale TEXT NOT NULL DEFAULT 'en-US',
  time_zone TEXT NOT NULL DEFAULT 'America/New_York',
  notify_hour_local SMALLINT CHECK (notify_hour_local BETWEEN 0 AND 23),
  paused_until TIMESTAMPTZ,
  onboarding_step SMALLINT NOT NULL DEFAULT 0,
  onboarding_done BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ
);
```

wisdom_tracks

```
CREATE TABLE wisdom_tracks (  
  id          wisdom_track PRIMARY KEY,  
  display_name TEXT          NOT NULL,  
  blurb       TEXT          NOT NULL,  
  accent_hex  CHAR(7)       NOT NULL,  
  is_live     BOOLEAN       NOT NULL DEFAULT false,  
  sort_order  SMALLINT      NOT NULL DEFAULT 0  
);  
  
INSERT INTO wisdom_tracks (id, display_name, blurb, accent_hex, is_live, sort_order) VALUES  
  ('secular', 'Secular & Psychology', 'Evidence-based tools from relationship science.', '#378ADD',  
  true, 1),  
  ('biblical', 'Biblical', 'Scripture-grounded guidance for Christian couples.', '#7F77DD', true,  
  2),  
  ('vedic',   'Vedic & Dharmic', 'Dharmic wisdom for Hindu couples.', '#BA7517', false, 3),  
  ('quranic', 'Quranic', 'Quranic guidance for Muslim couples.', '#1D9E75', false, 4);
```

user_tracks

This table is the moat

Track selection is **per user, not per couple**. An interfaith couple has two different rows here. Shared prompts are generated from the intersection of both.

```
CREATE TABLE user_tracks (  
  user_id      UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  track_id     wisdom_track NOT NULL REFERENCES wisdom_tracks(id),  
  is_primary   BOOLEAN       NOT NULL DEFAULT true,  
  selected_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),  
  PRIMARY KEY (user_id, track_id)  
);
```

04 Pairing tables

couples

```
CREATE TABLE couples (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  paired_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
  time_zone_mode TEXT       NOT NULL DEFAULT 'shared',  
  current_streak INTEGER    NOT NULL DEFAULT 0,  
  longest_streak INTEGER    NOT NULL DEFAULT 0,  
  freezes_used_month INTEGER NOT NULL DEFAULT 0,  
  freeze_reset_at DATE      NOT NULL DEFAULT CURRENT_DATE,  
  last_prompt_date DATE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  dissolved_at TIMESTAMPTZ  
);
```

couple_members

```
CREATE TABLE couple_members (  
  couple_id  UUID          NOT NULL REFERENCES couples(id) ON DELETE CASCADE,  
  user_id    UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  role       member_role NOT NULL,  
  joined_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  left_at    TIMESTAMPTZ,  
  is_active  BOOLEAN      NOT NULL DEFAULT true,  
  PRIMARY KEY (couple_id, user_id)  
);  
CREATE UNIQUE INDEX one_active_couple_per_user  
  ON couple_members (user_id) WHERE is_active;
```

invites

```
CREATE TABLE invites (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  inviter_id  UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  couple_id   UUID          REFERENCES couples(id) ON DELETE SET NULL,  
  code        CHAR(6)       NOT NULL UNIQUE,  
  token       UUID          NOT NULL UNIQUE DEFAULT gen_random_uuid(),  
  method      invite_method NOT NULL,  
  status      invite_status NOT NULL DEFAULT 'pending',  
  invitee_email CITEXT,  
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),  
  expires_at  TIMESTAMPTZ   NOT NULL DEFAULT now() + INTERVAL '7 days',  
  accepted_at TIMESTAMPTZ,  
  nudge_count SMALLINT      NOT NULL DEFAULT 0,  
  last_nudged_at TIMESTAMPTZ  
);  
CREATE INDEX invites_pending_idx ON invites (inviter_id) WHERE status = 'pending';
```

05 Content tables

prompts

```
CREATE TABLE prompts (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  track_id    wisdom_track NOT NULL REFERENCES wisdom_tracks(id),  
  body        TEXT          NOT NULL,  
  category    TEXT          NOT NULL,  
  cultural_ctx TEXT          NOT NULL DEFAULT 'us-general',  
  day_index   SMALLINT,  
  sentiment_weight SMALLINT NOT NULL DEFAULT 0,  
  is_sensitive BOOLEAN      NOT NULL DEFAULT false,  
  reviewed_by TEXT,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX prompts_track_idx ON prompts (track_id, day_index);  
CREATE INDEX prompts_category_idx ON prompts (category);
```

prompt_localizations

```
CREATE TABLE prompt_localizations (  
  prompt_id UUID NOT NULL REFERENCES prompts(id) ON DELETE CASCADE,  
  locale    TEXT NOT NULL,  
  body      TEXT NOT NULL,  
  reviewed_by TEXT,  
  PRIMARY KEY (prompt_id, locale)  
);
```

06 Daily loop tables

prompt_replies

```
CREATE TABLE prompt_replies (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  couple_id   UUID          NOT NULL REFERENCES couples(id) ON DELETE CASCADE,  
  prompt_id   UUID          NOT NULL REFERENCES prompts(id),  
  user_id     UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  for_date    DATE          NOT NULL,  
  ciphertext  BYTEA         NOT NULL,  
  nonce       BYTEA         NOT NULL,  
  mood        SMALLINT      CHECK (mood BETWEEN 1 AND 5),  
  is_voice    BOOLEAN       NOT NULL DEFAULT false,  
  visibility   visibility   NOT NULL DEFAULT 'shared',  
  revealed_at TIMESTAMPTZ,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (couple_id, prompt_id, user_id)  
);  
CREATE INDEX replies_couple_date_idx ON prompt_replies (couple_id, for_date DESC);
```

journal_entries

```
CREATE TABLE journal_entries (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  couple_id   UUID          REFERENCES couples(id) ON DELETE CASCADE,  
  user_id     UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  entry_date  DATE          NOT NULL,  
  ciphertext  BYTEA         NOT NULL,  
  nonce       BYTEA         NOT NULL,  
  mood        SMALLINT      CHECK (mood BETWEEN 1 AND 5),  
  visibility  visibility    NOT NULL DEFAULT 'shared',  
  source      entry_source  NOT NULL DEFAULT 'journal',  
  safety_flagged BOOLEAN     NOT NULL DEFAULT false,  
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),  
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT now()  
);  
CREATE INDEX journal_user_date_idx ON journal_entries (user_id, entry_date DESC);
```

mood_logs

```
CREATE TABLE mood_logs (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  couple_id   UUID          REFERENCES couples(id) ON DELETE CASCADE,  
  logged_date DATE          NOT NULL,  
  mood        SMALLINT      NOT NULL CHECK (mood BETWEEN 1 AND 5),  
  note_cipher BYTEA,  
  note_nonce  BYTEA,  
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),  
  UNIQUE (user_id, logged_date)  
);
```

love_language_results

```
CREATE TABLE love_language_results (  
  user_id     UUID          PRIMARY KEY REFERENCES profiles(id) ON DELETE CASCADE,  
  primary_lang TEXT          NOT NULL,  
  secondary_lang TEXT,  
  scores      JSONB         NOT NULL,  
  taken_at    TIMESTAMPTZ   NOT NULL DEFAULT now()  
);
```

07 AI tables

ai_conversations

```
CREATE TABLE ai_conversations (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  couple_id   UUID          REFERENCES couples(id) ON DELETE CASCADE,  
  mode        TEXT          NOT NULL DEFAULT 'text',  
  title_cipher BYTEA,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  last_msg_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

ai_messages

```
CREATE TABLE ai_messages (  
  id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  conversation_id   UUID          NOT NULL REFERENCES ai_conversations(id) ON DELETE CASCADE,  
  role              TEXT          NOT NULL CHECK (role IN ('user','assistant')),  
  ciphertext        BYTEA          NOT NULL,  
  nonce             BYTEA          NOT NULL,  
  input_tokens      INTEGER,  
  output_tokens     INTEGER,  
  model             TEXT,  
  latency_ms        INTEGER,  
  safety_flagged    BOOLEAN       NOT NULL DEFAULT false,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX ai_msgs_conv_idx ON ai_messages (conversation_id, created_at);
```

ai_memory_embeddings

Couple memory — the defensible feature

Enables the coach to reference what happened weeks ago. Retrieval must be scoped to `user_id = requester OR visibility = 'shared'`. **Never retrieve a partner's private entries.**

```

CREATE TABLE ai_memory_embeddings (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  couple_id   UUID          REFERENCES couples(id) ON DELETE CASCADE,
  source_type entry_source NOT NULL,
  source_id   UUID          NOT NULL,
  embedding   VECTOR(1536) NOT NULL,
  visibility  visibility   NOT NULL DEFAULT 'shared',
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE INDEX memory_user_idx ON ai_memory_embeddings (user_id);
CREATE INDEX memory_couple_idx ON ai_memory_embeddings (couple_id);
CREATE INDEX memory_vec_idx
  ON ai_memory_embeddings USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

08 Insight, subscription, and safety tables

relationship_pulse

```

CREATE TABLE relationship_pulse (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  couple_id   UUID          NOT NULL REFERENCES couples(id) ON DELETE CASCADE,
  state       pulse_state  NOT NULL DEFAULT 'unknown',
  score       NUMERIC(4,3),
  computed_on DATE          NOT NULL,
  factors     JSONB         NOT NULL DEFAULT '{}',
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
  UNIQUE (couple_id, computed_on)
);

```

streaks

```

CREATE TABLE streaks (
  couple_id   UUID          PRIMARY KEY REFERENCES couples(id) ON DELETE CASCADE,
  current     INTEGER       NOT NULL DEFAULT 0,
  longest     INTEGER       NOT NULL DEFAULT 0,
  last_both_on DATE,
  freezes_left SMALLINT    NOT NULL DEFAULT 2,
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

```

subscriptions

```
CREATE TABLE subscriptions (  
  user_id          UUID          PRIMARY KEY REFERENCES profiles(id) ON DELETE CASCADE,  
  couple_id        UUID          REFERENCES couples(id) ON DELETE SET NULL,  
  entitlement        entitlement NOT NULL DEFAULT 'free',  
  store            TEXT          CHECK (store IN ('apple','google','stripe','manual')),  
  product_id       TEXT,  
  original_txn_id  TEXT,  
  expires_at       TIMESTAMPTZ,  
  trial_ends_at    TIMESTAMPTZ,  
  grace_until      TIMESTAMPTZ,  
  is_shared_from   UUID          REFERENCES profiles(id),  
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Shared entitlement rule — when either partner purchases, the webhook writes a row for **both** members with `is_shared_from` set to the purchaser. This is what implements "one subscription covers both partners."

safety_events

No content, ever

This table stores **no user text**. Category and action only. This is intentional: a safety log must never become a readable archive of someone's worst moment.

```
CREATE TABLE safety_events (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  category    TEXT          NOT NULL,  
  severity    SMALLINT      NOT NULL,  
  action      safety_action NOT NULL,  
  source      entry_source  NOT NULL,  
  source_id   UUID,  
  resources_shown TEXT[],  
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now()  
);  
CREATE INDEX safety_user_idx ON safety_events (user_id, created_at DESC);
```

notification_events

```
CREATE TABLE notification_events (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,  
  kind        TEXT          NOT NULL,  
  sent_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),  
  opened_at   TIMESTAMPTZ,  
  provider_id TEXT  
);  
CREATE INDEX notif_user_idx ON notification_events (user_id, sent_at DESC);
```

feature_flags

```
CREATE TABLE feature_flags (  
  key          TEXT PRIMARY KEY,  
  enabled      BOOLEAN      NOT NULL DEFAULT false,  
  rollout_pct  SMALLINT     NOT NULL DEFAULT 0 CHECK (rollout_pct BETWEEN 0 AND 100),  
  updated_at   TIMESTAMPTZ  NOT NULL DEFAULT now()  
);
```

09 Indexes

Index	Table	Serves
replies_couple_date_idx	prompt_replies	Today's prompt lookup, journal timeline
journal_user_date_idx	journal_entries	Journal scroll, export
invites_pending_idx	invites	Waiting room status (partial index)
memory_vec_idx	ai_memory_embeddings	IVFFlat cosine similarity for couple memory
one_active_couple_per_user	couple_members	Enforces one active couple (partial unique)
ai_msgs_conv_idx	ai_messages	Conversation history retrieval

10 Row Level Security policies

Release gate

RLS must be enabled on every table, and policies must be verified by automated tests in CI. A policy gap here is the single worst possible bug in this product.

```

ALTER TABLE profiles          ENABLE ROW LEVEL SECURITY;
ALTER TABLE couples           ENABLE ROW LEVEL SECURITY;
ALTER TABLE couple_members    ENABLE ROW LEVEL SECURITY;
ALTER TABLE invites           ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_tracks       ENABLE ROW LEVEL SECURITY;
ALTER TABLE prompt_replies    ENABLE ROW LEVEL SECURITY;
ALTER TABLE journal_entries    ENABLE ROW LEVEL SECURITY;
ALTER TABLE mood_logs         ENABLE ROW LEVEL SECURITY;
ALTER TABLE love_language_results  ENABLE ROW LEVEL SECURITY;
ALTER TABLE ai_conversations   ENABLE ROW LEVEL SECURITY;
ALTER TABLE ai_messages       ENABLE ROW LEVEL SECURITY;
ALTER TABLE ai_memory_embeddings  ENABLE ROW LEVEL SECURITY;
ALTER TABLE relationship_pulse  ENABLE ROW LEVEL SECURITY;
ALTER TABLE streaks           ENABLE ROW LEVEL SECURITY;
ALTER TABLE subscriptions     ENABLE ROW LEVEL SECURITY;
ALTER TABLE safety_events     ENABLE ROW LEVEL SECURITY;

-- Helper: is this user an active member of this couple?
CREATE OR REPLACE FUNCTION is_member_of(cid UUID)
RETURNS BOOLEAN LANGUAGE sql STABLE AS $$
    SELECT EXISTS (
        SELECT 1 FROM couple_members
        WHERE couple_id = cid AND user_id = auth.uid() AND is_active
    );
$$;

-- Own rows
CREATE POLICY own_profiles ON profiles
    FOR ALL USING (id = auth.uid());

CREATE POLICY own_tracks ON user_tracks
    FOR ALL USING (user_id = auth.uid());

CREATE POLICY own_journal ON journal_entries
    FOR ALL USING (user_id = auth.uid());

CREATE POLICY own_mood ON mood_logs
    FOR ALL USING (user_id = auth.uid());

CREATE POLICY own_safety ON safety_events
    FOR ALL USING (user_id = auth.uid());

-- Couple-scoped rows: shared only, private stays private
CREATE POLICY couple_replies ON prompt_replies
    FOR ALL USING (
        user_id = auth.uid()
        OR (is_member_of(couple_id) AND visibility = 'shared')
    );

CREATE POLICY couple_journal_shared ON journal_entries
    FOR SELECT USING (
        is_member_of(couple_id) AND visibility = 'shared'
    );

CREATE POLICY couple_pulse ON relationship_pulse

```

```

    FOR SELECT USING (is_member_of(couple_id));

CREATE POLICY couple_streaks ON streaks
    FOR SELECT USING (is_member_of(couple_id));

CREATE POLICY own_conversations ON ai_conversations
    FOR ALL USING (user_id = auth.uid());

CREATE POLICY own_messages ON ai_messages
    FOR ALL USING (
        EXISTS (
            SELECT 1 FROM ai_conversations c
            WHERE c.id = conversation_id AND c.user_id = auth.uid()
        )
    );

-- Critical: never retrieve partner's private memory
CREATE POLICY scoped_memory ON ai_memory_embeddings
    FOR SELECT USING (
        user_id = auth.uid()
        OR (couple_id IS NOT NULL AND is_member_of(couple_id) AND visibility = 'shared')
    );

CREATE POLICY invites_own ON invites
    FOR ALL USING (
        inviter_id = auth.uid()
        OR invitee_email = (SELECT email FROM auth.users WHERE id = auth.uid())
    );

```

11 API map

Route	Method	Tables touched
<code>/rest/v1/profiles</code>	GET PATCH	profiles
<code>/rest/v1/user_tracks</code>	GET POST	user_tracks, wisdom_tracks
<code>/rest/v1/prompts</code>	GET	prompts, prompt_localizations
<code>/rest/v1/prompt_replies</code>	GET POST	prompt_replies
<code>/rest/v1/journal_entries</code>	GET POST PATCH	journal_entries
<code>/rest/v1/mood_logs</code>	POST	mood_logs
<code>/rest/v1/relationship_pulse</code>	GET	relationship_pulse
<code>/rest/v1/sync</code>	GET POST	batched multi-table delta
<code>/functions/v1/pairing/create- invite</code>	POST	invites
<code>/functions/v1/pairing/redeem</code>	POST	invites, couples, couple_members, streaks
<code>/functions/v1/pairing/nudge</code>	POST	invites, notification_events
<code>/functions/v1/ai-coach</code>	POST	ai_conversations, ai_messages, ai_memory_embeddings
<code>/functions/v1/safety-scan</code>	POST	safety_events
<code>/functions/v1/pulse-compute</code>	CRON	mood_logs, prompt_replies → relationship_pulse
<code>/functions/v1/recap-weekly</code>	CRON	read-only + notification_events
<code>/functions/v1/webhook- revenuecat</code>	POST	subscriptions (writes both partners)
<code>/functions/v1/account/export</code>	POST	all user-scoped tables
<code>/functions/v1/account/delete</code>	DELETE	all user-scoped tables, 30d grace

12 Migrations and seed data

Principles

- All migrations are forward-only, numbered, and reversible via companion down-files.
- Never drop a column in the same release that stops writing it. Two-phase: stop writing, then drop.
- Encrypted column changes require a client migration path — plan a re-encrypt job, not an in-place alter.

Seed requirements for local and staging

Entity	Count	Purpose
wisdom_tracks	4	Two live at v0.1
prompts	180	90 days × 2 launch tracks
profiles	10	Paired, solo, pending, dormant scenarios
couples + members	4	Includes one interfaith pair with differing tracks
journal_entries	200	Pagination and export testing
ai_conversations	5	Memory retrieval testing

Critical seed scenario

Include a couple where **partner A is on the Biblical track and partner B is on the Secular track**. Every shared-prompt decision must be tested against this pair. If the intersection logic is wrong, it will show up here first.

Sample query — daily prompt for a couple with differing tracks

```
-- Prompts that appear in BOTH partners' tracks, not yet used by this couple
SELECT p.id, p.body, p.category
FROM prompts p
WHERE p.track_id IN (
  SELECT track_id FROM user_tracks
  WHERE user_id IN (
    SELECT user_id FROM couple_members
    WHERE couple_id = $1 AND is_active
  )
)
AND p.is_live = true
AND p.id NOT IN (
  SELECT prompt_id FROM prompt_replies WHERE couple_id = $1
)
GROUP BY p.id, p.body, p.category
HAVING COUNT(DISTINCT p.track_id) = 1
ORDER BY p.day_index
LIMIT 1;
```